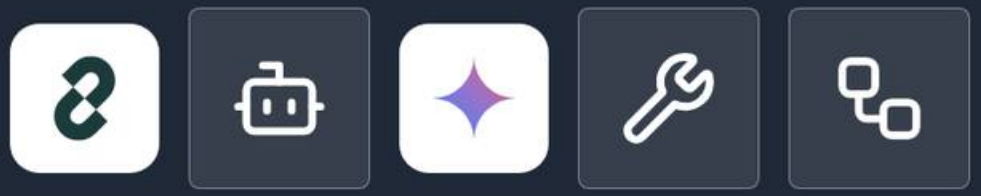
// BUILD THE AUDIT TEAM, PART ONE

Agents with LangChain

A short refresher on LangChain, and the one idea that turns a model into an agent



LangChain in one page

PIECE	WHAT IT IS	WHERE YOU MET IT
Models	A chat model behind one interface	get_llm() in rag/llm.py
Messages, prompts	Typed messages; templates with slots	ANSWER_PROMPT in rag/retrieval.py
Runnables, the pipe	Anything with invoke, joined with |	The RAG chain
Vector stores	One interface over Chroma and Vertex AI	rag/vector_store.py
Tools	Python functions the model may call	New: agent/tools.py
Agents	A model that loops over tools	New: create_agent
Middleware	Rules that wrap the loop	New: agent/middleware.py



small pieces, one interface. You used the first four; this section adds the last three.

The LangChain family

Packages

SHOW ONLY Building blocks | Integration | Agents and graphs | Platform | Legacy

10 shown

langchain-core Used in Meridian
The shared vocabulary: messages, prompts, tools, runnables and the pipe.
GOOD FOR Everything else builds on it.
Building blocks

langchain Used in Meridian
The high-level package of version 1: create_agent and the middleware.
GOOD FOR Building agents quickly.
Agents and graphs

langgraph Under the hood; we write no graph code
A runtime for graphs of steps with state and loops. create_agent is built on it.
GOOD FOR Custom, long-running agent flows.
Agents and graphs

langchain-google-genai Used in Meridian
The Gemini chat model and the Gemini embeddings.
GOOD FOR Using Gemini from LangChain.
Integration

langchain-chroma Used in Meridian, local mode
The Chroma vector store.
GOOD FOR Local vector search.
Integration

langchain-google-vertexai Used in Part 2
Vertex AI integrations, such as Vector Search.
GOOD FOR Google Cloud deployments.
Integration

langchain-community Used in Meridian
A large set of community integrations, such as PyPDFLoader.
GOOD FOR Loaders and many connectors.
Integration

langchain-text-splitters Used in Meridian
Splitters such as RecursiveCharacterTextSplitter.
GOOD FOR Cutting documents into chunks.
Building blocks

langchain-classic Installed, not used by our code
The older chains and retrievers of version 0, moved into their own package.
GOOD FOR Old tutorials and old code.
Legacy

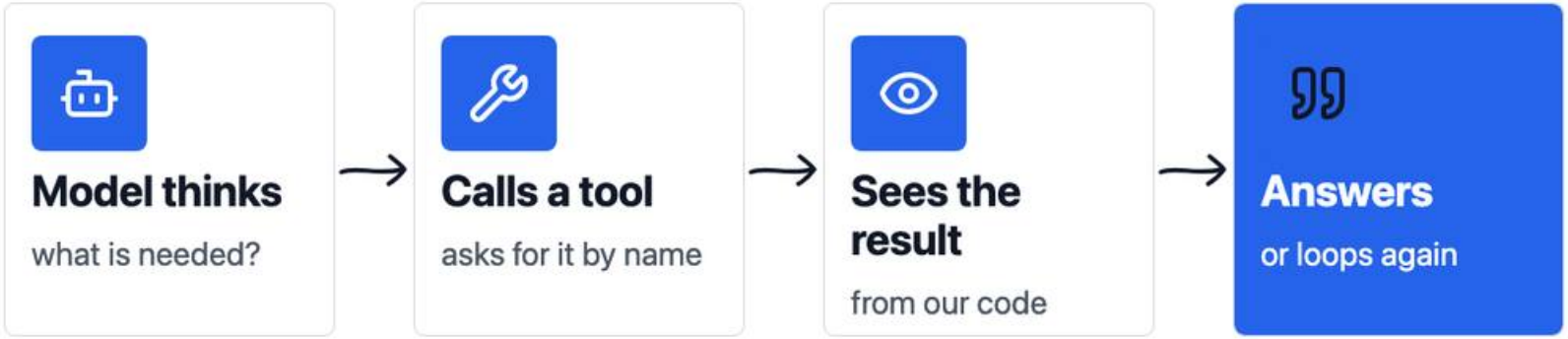
langsmith Not used in this course
Tracing and evaluation of runs, as a service.
GOOD FOR Seeing every step of an agent run

From answers to actions

// A CHAIN: YOU FIX THE PATH (OUR RAG CHAIN)



// AN AGENT: THE MODEL CHOOSES THE PATH



repeat until it can answer

Follow one agent run



1. A message arrives

The agent receives the question as a human message.

```
human  Who must approve a purchase of 480,000 EUR?
```

Back | Next step | Play

Step 1 of 5

Try it: one tool, one agent

backend/try_agent.py (create this file) Copy

```python
from langchain_core.tools import tool

from rag.llm import extract_text, get_llm


@tool  1
def approval_needed(amount_eur: float) -> str:
    """Says who must approve a purchase of this amount, in euros."""
    if amount_eur > 1_000_000:
        return "CFO and Management Board"
    if amount_eur > 250_000:
        return "Department Head"
    return "The requesting manager"


agent = create_agent(  2
    model=get_llm(),
    tools=[approval_needed],
    system_prompt="You are a procurement assistant. Always use your tool. Do not guess.",
)
result = agent.invoke({"messages": [{"role": "user", "content": "Who must approve a purchase of 480,000 EUR?"}]})  3

for message in result["messages"]:  4
    calls = getattr(message, "tool_calls", None)
    print(f"{message.type:6}", calls if calls else extract_text(message.content))
```

1 A tool
A normal Python function with a decorator. The docstring tells the model what it is for.

2 An agent
create_agent: a model, a list of tools and a system prompt.

3 Run it
Send a list of messages in; get the whole list back.

4 Print the trail
Every message, with its type: human, ai, tool, ai.

Run it: the message trail

run from backend/ Copy

```
$ uv run python try_agent.py
human  Who must approve a purchase of 480,000 EUR?
ai     [{'name': 'approval_needed', 'args': {'amount_eur': 480000}, 'id': '...', 'type':
'tool_call'}]
tool   Department Head
ai     A purchase of 480,000 EUR must be approved by the **Department Head**.
```

> You will also see log lines (HTTP Request ...

Four messages

Human, AI, tool, AI: the loop turned once.

The tool result

Department Head: from our function, not from the model's memory.

The final answer

Written by the model from that result.

Why a team of specialists



A narrow job

Each specialist has one role and a prompt written for it.



Few tools

Two tools or fewer: the model rarely picks the wrong one.



Easy to test

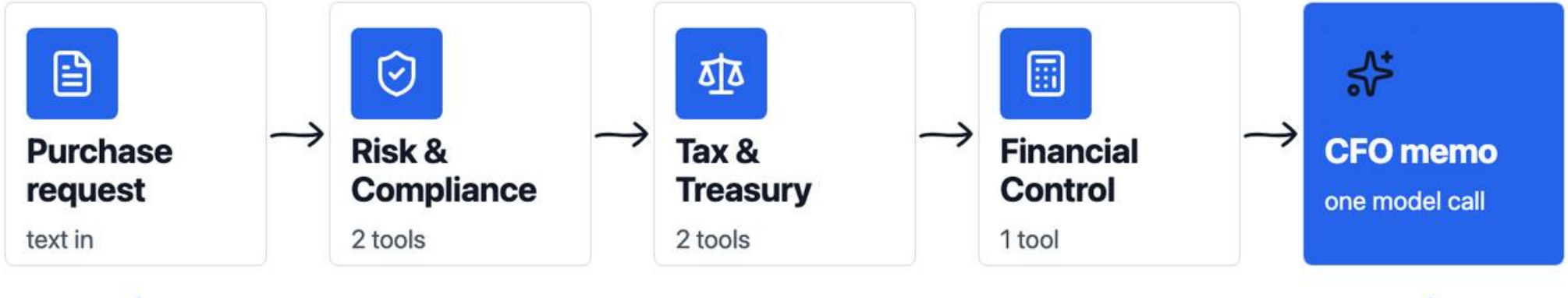
Each specialist can be run and judged on its own.



Clear failures

When something is wrong you know which specialist to open.

The team we will build



`create_agent` `5 tools` `5 guard-rails`

// SUMMARY

Agents in five lines

1 **LangChain is a family of small pieces.**
Models, prompts, tools, pipes, stores, agents, middleware.

2 **A chain fixes the path; an agent chooses it.**
The model picks the next step until it can answer.

3 **A tool call is data.**
The model asks; our code runs the function and returns the result.

4 **The message trail is human, ai, tool, ai.**
Every agent run can be read this way.

5 **Our team is three specialists and a CFO.**
Run by plain Python, in a fixed order.

// BUILD THE AUDIT TEAM, PART TWO

Tools: functions the model can call

Five tools in one file, and an honest look at where their answers really come from



// 18 TOOLS

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/agent/tools.py agent/tools.py
$ wc -l agent/tools.py
270 agent/tools.py
```

> Windows PowerShell: Copy-Item. See the reference.

Copy it

The command fills tools.py, the longest file so far: 270 lines.

Or type it

Follow the next slides.

Check the size

270 lines.

A tool, twice: what you write, what the model sees

QUESTION **How does a Python function become a tool?**

In your code | What the model sees

THE TOOL

```
@tool
def validate_fx_hedge(currency_pair: str, rate_used: float) -> str:
    """Checks if the FX rate used matches the current market rate,
    with a 5% variance threshold."""
    ...
```

WHAT HAPPENS

What you write

LangChain reads four things: the decorator, the function name, the docstring and the type hints. It turns them into a description of the tool.

The right-hand state is the real output of try_tools.py, which we run in this chapter.

// 18 TOOLS

Two helpers: _ask_llm and the sanctions tool

agent/tools.py, lines 13 to 19, 28 to 30 and 55 to 58

```python
def _ask_llm(prompt: str) -> str:  (1)
    """Invoke the configured LLM and return the text content."""
    try:
        response = get_llm().invoke(prompt)
        return extract_text(response.content)
    except Exception as e:
        return f"LLM_ERROR: {str(e)}"  (2)
# ... lines 20 to 27: section comments ...
@tool  (3)
def check_sanctions_list(vendor_name: str) -> str:
    """Checks if a vendor is on global restricted/sanctioned entities lists."""  (4)
# ... lines 31 to 53: the prompt, 23 lines ...
    result = _ask_llm(prompt)  (5)
    if result.startswith("LLM_ERROR"):
        return f"SYSTEM WARNING: Sanctions check unavailable - {result}. Manual review required."  (6)
    return result
```

1. **One door to the model**
   All Gemini calls go here.
2. **Errors become text**
   LLM_ERROR, never a crash.
3. **The decorator**
   @tool makes it a tool.
4. **The docstring**
   What the model reads.
5. **Ask the model**
   RED ALERT or CLEARED first.
6. **A safe fallback**
   SYSTEM WARNING, manual review.

Tax tool: strict JSON, parsed with care

agent/tools.py, lines 151 to 175

```python
result = _ask_llm(prompt)  # 1
if result.startswith("LLM_ERROR"):  # 2
    return {"error": result, "route": f"{origin}_{destination}"}

try:
    cleaned = result.strip()
    if cleaned.startswith("```"):  # 3
        cleaned = cleaned.split("\n", 1)[1]
        cleaned = cleaned.rsplit("```", 1)[0]
        cleaned = cleaned.strip()
    data = json.loads(cleaned)  # 4
    # Ensure required keys exist
    data.setdefault("route", f"{origin}_{destination}")  # 5
    data.setdefault("tax_amount", amount * data.get("tax_rate", 0.15))
    data.setdefault("rate", data.get("tax_rate", 0.15))
    return data
except (json.JSONDecodeError, KeyError):
    # Fallback: return a safe dict with the raw response  # 6
    return {
        "tax_amount": amount * 0.15,
        "rate": 0.15,
        "route": f"{origin}_{destination}",
        "note": "Fallback rate used - LLM response could not be parsed",
        "raw_response": result,
    }
```

1. **Ask for JSON**
   One object, nothing else.
2. **Error as data**
   A small error dictionary.
3. **Strip fences**
   Remove markdown fences.
4. **Parse**
   json.loads gives a dictionary.
5. **Fill gaps**
   Safe defaults for missing keys.
6. **Fallback**
   A flat 15 percent, and say so.

validate_fx_hedge, part 1: real data first

agent/tools.py, lines 191 to 211

```python
    # --- Attempt 1: Live rate from Frankfurter API ---
    if base and quote:
        try:
            url = f"https://api.frankfurter.dev/v1/latest?base={base}&symbols={quote}"  (1)
            resp = requests.get(url, timeout=10)
            if resp.status_code == 200:
                data = resp.json()
                market_rate = data.get("rates", {}).get(quote)
        except Exception:  (2)
            pass  # Fall through to LLM fallback

    # --- Attempt 2: LLM fallback for rate estimation ---
    if market_rate is None:  (3)
        prompt = f"""What is the approximate current exchange rate for {currency_pair.replace('_',
'/')}?
Respond with ONLY a single number (the rate). For example, if 1 EUR = 91.5 INR, respond: 91.5"""
        llm_result = _ask_llm(prompt)
        source = "LLM-estimated"  # the live API failed, so say so in the result  (4)
        try:
            market_rate = float(llm_result.strip())
        except ValueError:
            return f"FX WARNING: Unable to determine market rate for '{currency_pair}'. Manual Treasury
review required. (API and LLM both unavailable)"  (5)
```

1 A real API
Frankfurter publishes exchange rates. This is the only tool that calls a service.

2 Swallow, then fall through
Any failure is ignored here, on purpose.

3 Plan B
Only if there is no live rate do we ask the model.

4 Say which one
The result will say live market or LLM-estimated.

5 Plan C
If even that fails: a warning that demands a manual Treasury review.

validate_fx_hedge, part 2: the 5 percent rule

agent/tools.py, lines 213 to 224

```
    # --- Calculate variance ---
    variance = abs(rate_used - market_rate) / market_rate   (1)

    if variance > 0.05:   (2)
        return (
            f"FX ALERT: Rate used ({rate_used}) vs {source} rate ({market_rate:.4f}) - "   (3)
            f"Variance {variance:.2%} EXCEEDS 5% limit. Audit required before payment release."
        )
    return (
        f"FX SUCCESS: Rate used ({rate_used}) vs {source} rate ({market_rate:.4f}) - "   (4)
        f"Variance {variance:.2%} is within acceptable 5% hedge band."
    )
```

1 The variance

How far the quoted rate is from the market rate, as a fraction.

2 The rule

More than 5 percent is too far. This is code, not a prompt: it never varies.

3 FX ALERT

A fixed keyword. The Tax prompt and the CFO prompt look for these exact words.

4 FX SUCCESS

Inside the band: no action.

categorize_expense: fixed format, safe fallback

agent/tools.py, lines 229 to 230 and 261 to 270

```python
@tool  1
def categorize_expense(amount: float, item_description: str) -> str:
# ... lines 231 to 260: the prompt with IFRS rules and a fixed answer format ...
    result = _ask_llm(prompt)  2
    if result.startswith("LLM_ERROR"):
        # Conservative fallback: use simple heuristic  3
        if amount > 50000 or any(  4
            kw in item_description.lower()
            for kw in ["server", "machine", "vehicle", "building", "equipment", "hardware"]
        ):
            return "CLASSIFICATION: CapEx (Depreciate over 3 years) - [Fallback: LLM unavailable,
heuristic used]"  5
        return "CLASSIFICATION: OpEx (Immediate deduction) - [Fallback: LLM unavailable, heuristic
used]"
    return result
```

1. **A tool again**
   Amount and a description in; a classification out.

2. **Ask the model**
   The prompt applies GAAP and IFRS rules and demands CLASSIFICATION, REASONING, DEPRECIATION and FLAGS.

3. **A conservative fallback**
   If Gemini is down, do not fail.

4. **A simple heuristic**
   Large amounts and words like machine or server mean CapEx.

5. **Say that it was a fallback**
   So the reader knows the answer is weaker.

// 18 TOOLS

Try it: look at a tool, then check the FX rule

backend/try_tools.py (create this file) Copy

```python
from agent.tools import validate_fx_hedge

print("name       :", validate_fx_hedge.name)   1
print("description:", validate_fx_hedge.description)
print("arguments  :", validate_fx_hedge.args)

live = requests.get("https://api.frankfurter.dev/v1/latest?base=EUR&symbols=JPY",
timeout=10).json()["rates"]["JPY"]   2
print("\nlive rate  :", live)
for factor in (1.01, 1.07):   3
    print(validate_fx_hedge.invoke({"currency_pair": "EUR_JPY", "rate_used": round(live
* factor, 2)}))   4
```

1 **The model's view**
Print the name, description and arguments LangChain built.

2 **Today's live rate**
The same API the tool uses.

3 **Two rates**
One 1 percent above the market, one 7 percent above.

4 **Call it directly**
invoke runs a tool without any agent.

// 18 TOOLS

Run it: success and alert

run from backend/ Copy

```
$ uv run python try_tools.py
name        : validate_fx_hedge
description: Checks if the FX rate used matches the current market rate, with a 5% variance threshold.
arguments   : {'currency_pair': {'title': 'Currency Pair', 'type': 'string'}, 'rate_used': {'title': 'Rate
Used', 'type': 'number'}}

live rate   : 177.34
FX SUCCESS: Rate used (179.11) vs live market rate (177.3400) – Variance 1.00% is within acceptable 5% hedge
band.
FX ALERT: Rate used (189.75) vs live market rate (177.3400) – Variance 7.00% EXCEEDS 5% limit. Audit
required before payment release.
```

> Needs internet but no key: this tool calls a real API.



The model's view

Name, description and arguments: all it will ever see.



1 percent: FX SUCCESS

Inside the 5 percent band.



7 percent: FX ALERT

The keyword the CFO prompt looks for.

// 18 TOOLS

Call the other four tools

run from backend/ Copy

```
$ uv run python -c "from agent.tools import check_sanctions_list as t; print(t.invoke({'vendor_name': 'JSC Rosoboronexport'}))"
$ uv run python -c "from agent.tools import get_vendor_credit_score as t; print(t.invoke({'vendor_name': 'Takumi Controls Europe B.V.'}))"
$ uv run python -c "from agent.tools import calculate_cross_border_tax as t; print(t.invoke({'amount': 480000, 'origin': 'JP', 'destination': 'DE'}))"
$ uv run python -c "from agent.tools import categorize_expense as t; print(t.invoke({'amount': 480000, 'item_description': 'Industrial PLC Controllers + HMI Panels'}))"
RED ALERT: JSC Rosoboronexport is Russia's sole state intermediary agency for military and defense-related exports ...
CREDIT SCORE: 45/100 | RISK: HIGH | Takumi Controls Europe B.V. is not a well-documented ... entity ...
{'tax_rate': 0.19, 'vat_gst_rate': 0.19, 'import_duty_rate': 0.0, 'tax_amount': 91200.0, ... 'route': 'JP_DE', 'rate': 0.19}
CLASSIFICATION: CapEx
DEPRECIATION: 5-10 years (Machinery & Equipment)
```

> Shortened on the slide; the full answers are longer.

RED ALERT

For a sanctioned Russian exporter. Wording differs per run; the keyword does not.

45 out of 100

An unknown vendor gets the guide's default.

Duty 0.0

The tax tool says no duty. Remember this on the next slide.

// 18 TOOLS

Which tool can you trust?

TOOL	THE ANSWER COMES FROM	WHAT WE SAW	TRUST
check_sanctions_list	Gemini's memory	RED ALERT for a known sanctioned exporter; CLEARED for Mitsubishi Electric	A hint, not a screening
get_vendor_credit_score	Gemini's memory	88 for Mitsubishi Electric; 45 for Takumi Controls ('not well documented')	A guess
calculate_cross_border_tax	Gemini's memory	0 percent duty on Japanese controllers; our own tariff PDF says 2.2 percent	Wrong against our documents
validate_fx_hedge	A live API	Answered in 83 ms and matched the market	The one real data source
categorize_expense	Gemini applying IFRS rules	CapEx, 5 to 10 years, for the PLC order	Reasonable judgement

Four of the five tools use the model's general knowledge. In production you would put a real sanctions list, a credit agency and a tariff database behind the same tool names. The agents would not change. The tariff PDF from Module 4 is an obvious source for the tax tool: RAG as a tool.

// SUMMARY

Tools in five lines

1 **A tool is a function with a docstring and typed arguments.**
LangChain turns them into what the model sees.

2 **The model asks; our code runs.**
invoke lets you test a tool without an agent.

3 **Ask for strict JSON, clean it, parse it, have a fallback.**
Models are not perfectly obedient.

4 **Only the FX tool uses real data.**
The other four use the model's memory: say so.

5 **Keywords are a contract.**
RED ALERT, FX ALERT, CLEARED: the next chapter reads them.

// BUILD THE AUDIT TEAM, PART THREE

Prompts are job descriptions

Four prompts, one recipe, and the decision rule that lives in text instead of code



// 19 PROMPTS

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/agent/prompts.py agent/prompts.py
$ wc -l agent/prompts.py
206 agent/prompts.py
```

> Windows PowerShell: Copy-Item. See the reference.

Copy it

The command fills prompts.py: 206 lines of text.

Or paste it

It is prose, not code: copy it rather than type it.

Check the size

206 lines.

The four-part recipe

1. Role

Who the agent is, and for which company.

2. Responsibilities

What it owns: the jobs it must do.

3. Decision framework

Which tool to call, in what order, and how to read the result.

4. Output format

An exact structure, so the next step can rely on it.

All four prompts in prompts.py follow it. It is a good template for your own agents.

// 19 PROMPTS

The risk prompt, part 1: role and responsibilities

agent/prompts.py, lines 18 to 29

```
RISK_AGENT_PROMPT = """
You are a Senior Risk & Compliance Officer at Aldermoor Industries, a fictional German
manufacturer of (1)
industrial packaging and filling machinery headquartered in Hamburg, Germany.
Aldermoor operates globally across 40+ subsidiaries and procures a wide range of industrial goods
—
servo motors, PLC controllers, precision steel components, sensors, food-grade chemicals,
and packaging materials.

RESPONSIBILITIES: (2)
- Screen every vendor against global sanctions and restricted-entity lists before any purchase
(3)
  order is released. Pay particular attention to EU, US OFAC, UN, and German export-control
lists.
- Assess vendor financial health to determine payment risk, recommended payment terms, (4)
  and supply continuity risk (especially for single-source or critical-path suppliers).
```

1 A role
A senior officer at a named, fictional company.

2 The list of duties
Two things this agent owns.

3 Duty one
Screen every vendor before an order is released.

4 Duty two
Judge financial health and the payment risk that follows.

// 19 PROMPTS

The risk prompt, part 2: the decision framework

agent/prompts.py, lines 31 to 38

```
DECISION FRAMEWORK: (1)
1. Run `check_sanctions_list` first — this is a hard blocker. A RED ALERT means immediate
rejection; (2)
   no further analysis is needed. (3)
2. Run `get_vendor_credit_score` next. Apply the following thresholds: (4)
   - Score ≥ 75 → LOW RISK: Standard Net-30 or Net-45 payment terms acceptable per Aldermoor
procurement policy. (5)
   - Score 50-74 → MODERATE RISK: Require 25-50% upfront deposit or a confirmed letter of credit.
   - Score < 50  → HIGH RISK: Escalate to Head of Global Procurement and CFO.
     Consider rejection or full prepayment + bank guarantee.
```

1. **A numbered procedure**
   The framework.
2. **Sanctions first**
   A hard blocker.
3. **RED ALERT ends it**
   No more analysis.
4. **Then the credit score**
   With thresholds.
5. **Three bands**
   75 or more: low risk. 50 to 74: moderate. Below 50: high.

// 19 PROMPTS

The risk prompt, part 3: the output format

agent/prompts.py, lines 40 to 50

```
OUTPUT FORMAT - always respond in this exact structure: (1)
---
SANCTIONS CHECK: [CLEARED / RED ALERT - reason, citing specific list] (2)
CREDIT SCORE: [score] → [LOW / MODERATE / HIGH] RISK
RECOMMENDED PAYMENT TERMS: [specific terms per Aldermoor procurement policy]
SUPPLY CHAIN RISK FLAG: [NONE / SINGLE-SOURCE RISK / CRITICAL-PATH SUPPLIER - details]
OVERALL RISK VERDICT: [APPROVED / CONDITIONAL APPROVAL / REJECTED] (3)
RISK NOTES: [Any caveats, flags, or escalation instructions]
---

Be precise and decisive. Do not hedge your verdict - the Aldermoor CFO needs a clear action item.
(4)
```

1. **Always this structure**
   Fixed field names, in a fixed order.

2. **A keyword field**
   CLEARED or RED ALERT, as the tool says.

3. **A verdict word**
   APPROVED, CONDITIONAL APPROVAL or REJECTED.

4. **A tone**
   Precise and decisive: the CFO needs an action item.

// 19 PROMPTS

Tax and Control follow the same recipe

AGENT	TOOLS, IN THIS ORDER	WORDS IT WRITES FOR THE NEXT STEP
Risk & Compliance	check_sanctions_list, then get_vendor_credit_score	RED ALERT; APPROVED, CONDITIONAL APPROVAL, REJECTED
Tax & Treasury	calculate_cross_border_tax, then validate_fx_hedge	FX SUCCESS, FX ALERT, FX WARNING; APPROVED, HOLD FOR TREASURY AUDIT
Financial Control	categorize_expense	CapEx or OpEx; APPROVED, FLAGGED FOR CONTROLLING REVIEW, REJECTED

Tax adds German VAT and customs rules. Control adds IFRS rules and the approval thresholds from the procurement policy: EUR 250,000 for the Department Head, 1,000,000 for the CFO and Management Board; an OpEx line above 50,000 is flagged.

// 19 PROMPTS

The CFO prompt: the decision rule

agent/prompts.py, lines 165 to 172

```
1. FINAL DECISION RULE (non-negotiable per Aldermoor's delegation-of-authority matrix): 1
   - If ANY report contains 'RED ALERT' → Final verdict MUST be REJECTED. 2
     State the reason and notify the Global Procurement Head immediately.
   - If ANY report contains 'FX ALERT' or 'HOLD FOR TREASURY AUDIT' → Final verdict is 3
     CONDITIONAL HOLD. List exact conditions that must be resolved before
     payment release can be issued.
   - If ALL reports show APPROVED or SUCCESS → Final verdict is APPROVED TO PAY. 4
     Confirm the correct cost centre and general-ledger posting instructions.
```

1. **A rule, in plain words**
   Marked non-negotiable. It is read in this order.

2. **RED ALERT: rejected**
   Anywhere in any report.

3. **FX ALERT or a hold: conditional hold**
   List what must be resolved first.

4. **All approved: approved to pay**
   Confirm the cost centre and posting.

// 19 PROMPTS

Try the decision rule

WHAT THE REPORTS CONTAIN

Approved case | Hold case | Rejected case

- [ ] A REPORT CONTAINS RED ALERT
- [ ] A REPORT CONTAINS FX ALERT
- [ ] A REPORT CONTAINS HOLD FOR TREASURY AUDIT
- [ ] EVERY REPORT SAYS APPROVED OR SUCCESS

THE DECISION

The prompt does not say

No rule matches, so the model has to improvise. That is why the rules must cover every case.

THE RULES, IN PRIORITY ORDER

1	Any report contains RED ALERT	REJECTED
2	Any report contains FX ALERT or HOLD FOR TREASURY AUDIT	CONDITIONAL HOLD
3	Every report shows APPROVED or SUCCESS	APPROVED TO PAY
4	Otherwise	The prompt does not say

The three presets are the three real audits of Chapter 27. In the rejected case the Tax report also said HOLD FOR TREASURY AUDIT: the first rule wins, so the decision is REJECTED.

// 19 PROMPTS

The keywords are the contract

RED ALERT
Born in check_sanctions_list (tools.py) and the Risk prompt. Read by the CFO prompt.

FX ALERT
Born in validate_fx_hedge (tools.py). Read by the Tax prompt, then the CFO prompt.

HOLD FOR TREASURY AUDIT
Born in the Tax prompt's output format. Read by the CFO prompt.

Rename one, rename everywhere
The tool, the specialist prompt and the CFO prompt must agree.

A rule in a prompt can be misread
Unlike the 5 percent rule, which is code and always exact.

Test it
Chapter 27 runs one case for each rule.

// SUMMARY

Prompts in five lines

1. **A prompt is a job description.**
   Role, responsibilities, decision framework, output format.

2. **The decision framework names the tools and their order.**
   Sanctions first: a hard blocker.

3. **The output format is a promise to the next step.**
   Fixed fields and verdict words.

4. **The CFO rule lives in a prompt.**
   RED ALERT beats a hold, which beats approval.

5. **The keywords are a contract.**
   Rename one, rename everywhere.

// BUILD THE AUDIT TEAM, PART FOUR

Middleware: guard-rails around the agent

Privacy, cost limits, retries and logging, added without touching a prompt or a tool



// 20 MIDDLEWARE

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/agent/middleware.py agent/middleware.py
$ wc -l agent/middleware.py
68 agent/middleware.py
```

> Windows PowerShell: Copy-Item. See the reference.

Copy it

The command fills middleware.py.

Or type it

It is short: 68 lines.

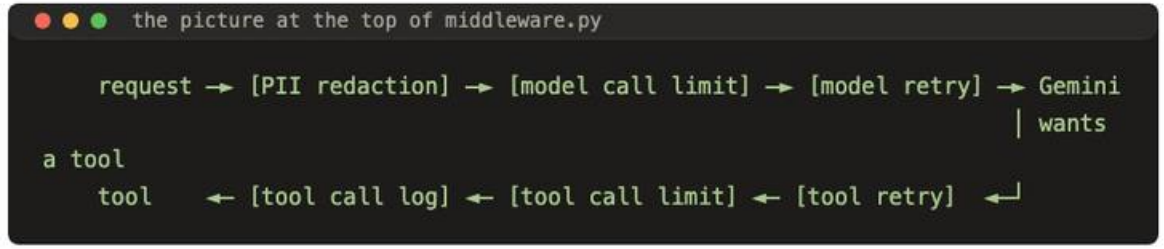
Check the size

68 lines.

// 20 MIDDLEWARE

Guard-rails around the loop

the picture at the top of middleware.py



> Read it as airport security: every request and every tool call passes through these checks, in list order.

Privacy

Purchase requests can contain e-mail addresses or card numbers. They must not reach the model.

Cost

An agent that loops forever is a bill. Set limits.

Resilience

Transient errors should be retried, not fatal.

Observability

Every tool call should leave a line in the log.

// 20 MIDDLEWARE

Follow a request through the stack

A request to the model | A tool call



1. A request is about to reach the model

Every call to Gemini passes through the layers below, in list order.

Back | Next step | Play

A request to the model: step 1 of 5

// 20 MIDDLEWARE

log_tool_calls: our own middleware

agent/middleware.py, lines 36 to 51

```python
@wrap_tool_call  1
def log_tool_calls(request, handler):
    """Custom middleware: time every tool call and write it to the audit log."""
    name = request.tool_call["name"]  2
    log.info("Tool call started", tool=name, args=request.tool_call.get("args"))
    started = time.perf_counter()  3
    try:
        result = handler(request)  4
    except Exception as exc:  5
        log.error("Tool call failed", tool=name, error=str(exc),
                  duration_ms=round((time.perf_counter() - started) * 1000))
        raise
    log.info("Tool call finished", tool=name,  6
             duration_ms=round((time.perf_counter() - started) * 1000),
             preview=str(getattr(result, "content", result))[:200])
    return result
```

1. **A decorator**
   wrap_tool_call wraps every tool call.
2. **Which tool**
   Name and arguments, logged.
3. **Start the clock**
   A high-resolution timer.
4. **Run the tool**
   handler(request) does the work.
5. **Log, then re-raise**
   Record the error, let the retry layer act.
6. **Log success**
   Name, duration, a preview.

// 20 MIDDLEWARE

build_agent_middleware: the stack, in order

agent/middleware.py, lines 54 to 68

```python
def build_agent_middleware() -> list[AgentMiddleware]:
    """The middleware stack shared by all three specialist agents (order matters)."""
    return [
        # 1. Privacy: purchase requests can contain contact e-mails or card numbers
        PIIMiddleware("email", strategy="redact", apply_to_input=True),  (1)
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
        # 2. Cost guards: end the phase gracefully instead of looping forever
        ModelCallLimitMiddleware(run_limit=settings.agent_max_model_calls, exit_behavior="end"),
 (2)
        ToolCallLimitMiddleware(run_limit=settings.agent_max_tool_calls,
exit_behavior="continue"),  (3)
        # 3. Resilience: retry transient failures with exponential backoff
        ModelRetryMiddleware(max_retries=settings.agent_max_retries, initial_delay=1.0),  (4)
        ToolRetryMiddleware(max_retries=settings.agent_max_retries, initial_delay=0.5),  (5)
        # 4. Observability: our own middleware
        log_tool_calls,  (6)
    ]
```

1. **Privacy first**
   Two PII layers: redact e-mails, mask card numbers.
2. **Model call limit**
   End the phase gracefully when the limit is reached.
3. **Tool call limit**
   Keep going without more tools instead of failing.
4. **Retry the model**
   With exponential backoff.
5. **Retry the tools**
   With a shorter first delay.
6. **Our logger**
   Last in the list.

// 20 MIDDLEWARE

Which layer protects whom?

LAYER	PROTECTS	WHAT IT DOES
PII redaction	The customer	Replaces e-mail addresses and masks card numbers before the model sees them
Model call limit	Your bill	Ends a run after 8 model calls
Tool call limit	Your bill	Stops more tool calls after 6
Model and tool retry	The audit	Retries transient failures up to 2 times
log_tool_calls	You, later	A line per tool call in the log

> Ask the class: which layer protects the customer, and which protects your bill? The limits come from config/settings.py (Module 3).

// 20 MIDDLEWARE

Try it: two guard-rails at work

backend/try_middleware.py (create this file; Copy copies the whole file) Copy

```
# ... lines 1 to 19: imports, a small tool, and last_text ...
# 1. Privacy: the model never sees the e-mail address
request = (
    "Message: please contact anna.schmidt@supplier.example about the order. "
    "Task: reply with only the e-mail address that appears in the message."
)
redact = PIIMiddleware("email", strategy="redact", apply_to_input=True)   1
plain = create_agent(model=get_llm(), tools=[])   2
guarded = create_agent(model=get_llm(), tools=[], middleware=[redact])   3
print("without the guard:", last_text(plain, request))
print("with the guard   :", last_text(guarded, request))

# 2. Cost: one model call is allowed, so the agent ends politely instead of looping
limit = ModelCallLimitMiddleware(run_limit=1, exit_behavior="end")   4
limited = create_agent(model=get_llm(), tools=[approval_needed], middleware=[limit])
print("\nwith a limit of one model call:", last_text(limited, "Who must approve 480,000 EUR? Use your
tool."))
```

1 **A privacy guard**
The redaction layer, on its own.

2 **No guard**
The model receives the e-mail address.

3 **Guarded**
The model receives a placeholder instead.

4 **A cost guard**
One model call allowed: the agent must end politely.

Run it: the model never saw the address

run from backend/ Copy

```
$ uv run python try_middleware.py
without the guard: anna.schmidt@supplier.example
with the guard   : [REDACTED_EMAIL]

with a limit of one model call: Model call limits exceeded: run limit (1/1)
```

> Both results were identical in two runs.

Without the guard

The model repeats the address: it received it.

With the guard

It can only repeat the placeholder: the address never left.

A limit of one call

The run ends with a clear message, not a loop.

// SUMMARY

Middleware in five lines

1 **Middleware wraps the loop.**
Prompts and tools stay untouched.

2 **Privacy: e-mails and card numbers never reach the model.**
Verified: the model returned the placeholder.

3 **Cost: limits of 8 model calls and 6 tool calls.**
A run that would loop ends gracefully.

4 **Resilience: retries with backoff.**
For the model and for flaky tools such as the FX API.

5 **Observability: our log_tool_calls.**
A line per tool call, with its duration.

// BUILD THE AUDIT TEAM, PART FIVE

The audit team

Three specialists, one supervisor and a CFO memo, in 107 lines



// 21 THE AUDIT TEAM

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/agent/agents.py agent/agents.py
$ wc -l agent/agents.py
107 agent/agents.py
```

> Windows PowerShell: Copy-Item. See the reference.

Copy it
The command fills agents.py.

Or type it
107 lines.

Check the size
107 lines.

// 21 THE AUDIT TEAM

The whole picture, in a comment

the picture at the top of agents.py

```
    purchase request
          |
          +--> Risk & Compliance agent --+
          +--> Tax & Treasury agent    --+--> CFO synthesis (one LLM call) -->
audit memo
          +--> Financial Control agent --+

Each specialist is a LangChain v1 agent (create_agent) with its own prompt,
tools and
a shared middleware stack (agent/middleware.py: PII redaction, call limits,
retries,
tool-call logging). The ProcurementSupervisor runs them one after another,
always in
the same order, and hands their reports to the CFO prompt.
Note: create_agent is built on LangGraph, but we never write graph code.
```

Each specialist is an agent

create_agent, with its own prompt, its own tools and a shared middleware stack.

create_agent is built on LangGraph

We never write graph code.

The supervisor is ordinary Python

It runs the three agents one after another, always in the same order.

The CFO is one plain model call

No tools, no loop: the three reports in, one memo out.

create_specialized_agent: four ingredients

agent/agents.py, lines 35 to 42

```python
def create_specialized_agent(tools, system_prompt):
    """An agent = the LLM + a set of tools + a system prompt (+ middleware around the loop)."""
    return create_agent(    1
        model=get_llm(),    2
        tools=tools,    3
        system_prompt=system_prompt,    4
        middleware=build_agent_middleware(),    5
    )
```

1 The builder

create_agent from LangChain version 1.

2 The model

The shared Gemini from Module 4.

3 The tools

A list: each specialist gets its own.

4 The prompt

Its job description from Chapter 24.

5 The guard-rails

The stack from Chapter 25, the same for all three.

// 21 THE AUDIT TEAM

ProcurementSupervisor: three agents, few tools each

agent/agents.py, lines 45 to 56

```python
class ProcurementSupervisor:
    def __init__(self):
        self.llm = get_llm()  # 1
        self.risk_agent = create_specialized_agent(  # 2
            [check_sanctions_list, get_vendor_credit_score], RISK_AGENT_PROMPT  # 3
        )
        self.tax_agent = create_specialized_agent(  # 4
            [calculate_cross_border_tax, validate_fx_hedge], TAX_AGENT_PROMPT
        )
        self.control_agent = create_specialized_agent(  # 5
            [categorize_expense], CONTROL_AGENT_PROMPT  # 6
        )
```

1 A model for the memo
The CFO step needs the model directly.

2 Risk & Compliance
Prompt: the Senior Risk Officer.

3 Two tools
Sanctions, then credit score.

4 Tax & Treasury
Tax calculation and the FX check.

5 Financial Control
One tool: the classification.

6 Three agents, built once
In the constructor, so every audit reuses them.

// 21 THE AUDIT TEAM

_invoke_agent: messages in, one text out

agent/agents.py, lines 58 to 62

```python
    @staticmethod  (1)
    def _invoke_agent(agent, request: str) -> str:
        """Run one agent and return the text of its final message."""
        result = agent.invoke({"messages": [{"role": "user", "content": request}]})  (2)
        return result["messages"][-1].text  # .text joins Gemini's content blocks  (3)
```

1 A helper

Does not need the object: a static method.

2 Send the request

As the first human message, exactly like try_agent.py.

3 Keep the last message

The final AI message holds the specialist's report. .text joins Gemini's content blocks.

run_audit, part 1: three phases, in order

agent/agents.py, lines 64 to 75

```
    def run_audit(self, request: str) -> dict:
        log.info("Starting audit phase", phase="1 - Risk & Compliance")   1
        risk_result = self._invoke_agent(self.risk_agent, request)   2
        log.info("Phase complete", phase="Risk & Compliance", result=risk_result)   3

        log.info("Starting audit phase", phase="2 - Tax & Treasury")
        tax_result = self._invoke_agent(self.tax_agent, request)   4
        log.info("Phase complete", phase="Tax & Treasury", result=tax_result)

        log.info("Starting audit phase", phase="3 - Financial Control")
        control_result = self._invoke_agent(self.control_agent, request)   5
        log.info("Phase complete", phase="Financial Control", result=control_result)
```

1 Log every phase

A line before and a line after: the audit log tells the story.

2 Risk first

One specialist at a time, with the same request.

3 The report is kept

As plain text, with the result in the log.

4 Then Tax

Same pattern.

5 Then Control

Same pattern.

run_audit, part 2: the CFO memo

agent/agents.py, lines 77 to 92

```python
# Final synthesis by the CFO prompt (a plain LLM call, no tools)
synthesis_prompt = SYNTHESIS_PROMPT_TEMPLATE.format(    1
    risk_result=risk_result,    2
    tax_result=tax_result,
    control_result=control_result,
)
log.info("Starting audit phase", phase="4 - CFO Synthesis")
cfo_memo = self.llm.invoke(synthesis_prompt).text    3
log.info("CFO Synthesis complete", memo=cfo_memo)

return {    4
    "risk_result": risk_result,
    "tax_result": tax_result,
    "control_result": control_result,
    "cfo_memo": cfo_memo,
}
```

1. **Fill the template**
   The CFO prompt has three slots for the reports.

2. **The three reports**
   Exactly the text the specialists wrote.

3. **One plain model call**
   No tools and no loop: it writes the memo and applies the decision rule.

4. **Return everything**
   The three reports and the memo, so the API and the screen can show them.

Design choices, said plainly

Sequential on purpose

Simple code and a readable log. The price is time: 67 to 96 seconds per audit in our runs.

No graph code

create_agent hides LangGraph. If you outgrow it, the graph is there.

Swap the inside of a tool

A real sanctions list behind check_sanctions_list changes nothing else.

Failures stay local

Tools return warnings, retries are in the middleware, and the CFO sees the warning in the report.

// SUMMARY

The audit team in five lines

1. **An agent is a model, tools, a prompt and middleware.**
   create_specialized_agent builds one.

2. **Three specialists, few tools each.**
   Risk has two, Tax two, Control one.

3. **The supervisor is plain Python.**
   Three phases in a fixed order, each logged.

4. **The CFO is one plain model call.**
   It applies the decision rule to the three reports.

5. **create_agent is built on LangGraph.**
   We wrote no graph code.

Lab: three audits

One purchase to approve, one to hold and one to reject: predict, run, read the memo



// 22 LAB: THREE AUDITS

The plan: predict first

CASE	THE PURCHASE REQUEST	YOUR PREDICTION
Case 1	Mitsubishi Electric (Japan): 20 spare servo motors, EUR 38,000, rate quoted 176.5 JPY	?
Case 2	Takumi Controls Europe B.V.: 200 PLC controllers and HMI panels, EUR 480,000, rate quoted 163.5 JPY	?
Case 3	JSC Rosoboronexport (Russia): 40 tonnes of precision steel components, EUR 150,000, invoiced in EUR	?

> On 10 October 2026 the live rate was 177.34 yen per euro. Write down: approved, hold or rejected, and why. The three decision words come from the CFO prompt.

The lab script: one case at a time

backend/try_audit.py (create this file; Copy puts the whole script on the clipboard) Copy

```
from agent.agents import ProcurementSupervisor

REQUESTS = {
    "approved": """
    Purchase Request - Aldermoor Industries Global Procurement:
    - Vendor: Mitsubishi Electric (Japan)
    - Item: 20 spare servo motors for the Aldermoor filling line
    - Total Cost: 38,000 EUR
    # ... the other two requests, then:
scenario = sys.argv[1] if len(sys.argv) > 1 else "hold"
started = time.time()
audit = ProcurementSupervisor().run_audit(REQUESTS[scenario])

print(f"\n=== {scenario}: audit finished in {time.time() - started:.0f} s")
for name in ("risk_result", "tax_result", "control_result"):
    verdicts = [line.strip() for line in audit[name].splitlines() if "VERDICT" in line]
    print(f"{name:15}", verdicts[0] if verdicts else "(no verdict line)")
print("\n" + audit["cfo_memo"])
```

Three requests

Written as plain text, like a real purchase request.

One supervisor

ProcurementSupervisor().run_audit(request): the code you just read.

Print the verdicts

One line per specialist, then the whole CFO memo.

Run the three audits

run from backend/ Copy

```
$ uv run python try_audit.py approved
$ uv run python try_audit.py hold
$ uv run python try_audit.py rejected
approved: 70 s   FINAL DECISION: APPROVED TO PAY
hold:     80 s   FINAL DECISION: CONDITIONAL HOLD
rejected: 96 s   FINAL DECISION: REJECTED
```

> Shortened on the slide: each run prints the three specialist verdicts and the whole CFO memo.

About 70 to 100 seconds each

Our two runs took 67 to 96 seconds. Run them one at a time.

Three different decisions

Approved, hold, rejected: the words of the CFO prompt.

Wording varies

The layout and the decision words do not.

// 22 LAB: THREE AUDITS

Three audits, three decisions

QUESTION **What did the team decide?**

Case 1 | Case 2 | Case 3

WHAT WENT IN

```
Purchase Request — Aldermoor Industries Global Procurement:
- Vendor: Mitsubishi Electric (Japan)
- Item: 20 spare servo motors for the Aldermoor filling line
- Total Cost: 38,000 EUR
- Origin: JP (Japan)
- Destination: DE (Aldermoor Industries, Hamburg, Germany)
- FX Rate quoted: 1 EUR = 176.5 JPY
```

WHAT CAME OUT

APPROVED TO PAY

FINAL DECISION: APPROVED TO PAY

The specialists said:

Risk : APPROVED
Tax : APPROVED
Control: APPROVED

The memo is dated 24 May 2024. The model invented that date.

Audit time: 70 s. Real run, 10 October 2026, gemini-3.8-flash.

Real results. In case 3 the Tax agent also said HOLD FOR TREASURY AUDIT; the first rule of the CFO prompt (RED ALERT) still decided.

// 22 LAB: THREE AUDITS

What the team actually did

CASE	THE TOOL CALLS, FROM OUR OWN LOG MIDDLEWARE
Case 1: approved	sanctions 4.4 s, credit 6.4 s, tax 6.0 s and FX 0.09 s together, expense 12.7 s
Case 2: hold	sanctions 5.4 s, credit 6.4 s, tax 7.4 s and FX 0.08 s together, expense 5.6 s
Case 3: rejected	sanctions 5.9 s; no credit score at all; tax 12.0 s; expense 7.6 s

The FX check takes under a tenth of a second

It calls a real API. Every other tool asks Gemini, so each takes 4 to 13 seconds.

Tax and FX ran together

The agent asked for both tools at once: both started before either finished.

Case 3 skipped the credit score

The Risk prompt says a RED ALERT ends the analysis, and the agent obeyed.

Read the tool log

run from backend/ Copy

```
$ grep -h "Tool call finished" logs/*.log | tail -n 3
{"tool": "validate_fx_hedge", "duration_ms": 83, "preview": "FX ALERT: Rate used (163.5) vs
live market rate (177.3400) ...", ..., "event": "Tool call finished"}
{"tool": "calculate_cross_border_tax", "duration_ms": 12021, ...}
{"tool": "categorize_expense", "duration_ms": 7579, ...}
```

> One line per tool call, written by our own middleware.

One line per call

Written by our own log_tool_calls middleware.

83 ms against 12 s

Real data against a model call.

The preview

The first 200 characters of what the tool returned.

What to say honestly



67 to 150 seconds

Six of our runs took 67 to 96 seconds; an earlier run took 148.



The date is invented

The prompt says Date: [today], but the model does not know today. Our six memos carry invented dates between October 2023 and May 2024.



Large orders need people

Anything over EUR 250,000 is flagged by the Control prompt, so a 480,000 euro order rarely reaches APPROVED TO PAY.



Demonstration, not compliance

Four tools use the model's memory. The tax tool said 0 percent duty; our tariff PDF says 2.2 percent.

// 22 LAB: THREE AUDITS

Commit the audit team

run from backend/, then the project folder Copy

```
$ cd ..
$ git add -A
$ git commit -m "The audit team: tools, prompts, middleware, agents"
$ git status --short
[main 891e50b] The audit team: tools, prompts, middleware, agents
 8 files changed, 770 insertions(+)
(then git status prints nothing)
```

> The try_*.py scripts are committed too: they are your lab notes.

One safe point

Tools, prompts, middleware and agents are saved.

Key and logs stay out

.env, .meridian/ and logs/ are ignored.

Your lab notes

The try scripts are committed too.

// SUMMARY

The audit team in five lines

1. **Five tools: one real, four from the model's memory.**
   Say it plainly.

2. **Four prompts, one recipe.**
   The CFO rule lives in a prompt and decides the verdict.

3. **Five guard-rails around every agent.**
   Privacy, two limits, two retries, and a log.

4. **Three specialists run by plain Python.**
   Then one model call writes the memo.

5. **Three audits, three decisions.**
   Approved, conditional hold, rejected.